Đề tài:

Xây dựng hệ thống đếm lưu lượng & phân loại phương tiện giao thông

GV: **NCS. ThS. Cáp Phạm Đình Thăng**

TP. Hồ Chính Minh, tháng 10 năm 2026

Cấu trúc trình bày

1. Giới thiệu bài toán, định nghĩa bài toán, mục tiêu
2. Các nghiên cứu liên quan, câu hỏi nghiên cứu
3. Phương pháp thực hiện
4. Dữ liệu và đánh giá
5. Kết quả
6. Kết luận và hướng phát triển
7. Tài liệu tham khảo

Bối cảnh và vấn đề nghiên cứu

- Giao thông đô thị Việt Nam có mật độ cao, dòng xe hỗn hợp và xe máy chiếm ưu thế.
- Đếm thủ công tốn nhân lực, khó duy trì liên tục; cảm biến chuyên dụng khó mở rộng.
- Camera giám sát hiện hữu tạo cơ hội tự động hóa phát hiện, theo dõi và thống kê lưu lượng.
- Mô hình huấn luyện trên dữ liệu quốc tế suy giảm hiệu năng khi áp dụng trực tiếp vào bối cảnh Việt Nam.



Định nghĩa bài toán

ĐẦU VÀO

Video / camera cố định

XỬ LÝ

Phát hiện → Theo dõi → Đếm

ĐẦU RA

Phương tiện theo loại, hướng di chuyển

Mục tiêu tổng quát: xây dựng hệ thống tự động phát hiện, phân loại, theo dõi, đếm và trực quan hóa phương tiện giao thông theo thời gian thực.

Mục tiêu nghiên cứu

01 Nhận diện 4 lớp

Ô tô, xe máy, xe tải, xe buýt

02 Chọn chiến lược tinh chỉnh

Thích nghi dữ liệu giao thông Việt Nam



03 Theo dõi và đếm

Gán ID, xác định lượt cắt vạch và hướng

04 Đánh giá toàn diện

Độ chính xác, tổng quát hóa và thời gian thực

Nghiên cứu liên quan và khoảng trống

Công trình	Cách tiếp cận	Trọng tâm
Ribeiro & Hirata	YOLO + Visual Rhythm	Đếm nhanh, phù hợp dòng một chiều
Khan & Rishe	YOLOv10-S + chưng cất	Khung phát hiện, theo dõi, đếm hợp nhất
Nguyễn Mạnh Cường và cs.	YOLO + SORT	Nhấn mạnh đặc thù xe máy Việt Nam
Nguyễn Văn Viết	YOLOv7 + StrongSORT	Hiệu năng còn hạn chế trên dữ liệu thực tế

Khoảng trống: vai trò của miền trung gian chưa được kiểm định; thiếu đánh giá chéo miền; khoảng cách từ mô hình thử nghiệm đến hệ thống vận hành chưa được làm rõ.

Câu hỏi nghiên cứu

Q1 Tinh chỉnh trên dữ liệu TP.HCM cải thiện nhận diện như thế nào?

Q2 CanTho có thực sự hữu ích như miền trung gian?

Q3 Tinh chỉnh ảnh hưởng thế nào đến khả năng tổng quát chéo miền?

Q4 Hệ thống hoàn chỉnh có đạt độ chính xác và tốc độ thời gian thực?

Phạm vi nghiên cứu

4 lớp

car • motorbike • truck • bus

3 miền dữ liệu

Vehicles-COCO • CanTho • AIC-HCMC-2020

1 nguồn thu hình

Góc quan sát cố định

Chuỗi xử lý

YOLOv8n • ByteTrack • Line-crossing

Đầu ra

SQLite • Dashboard Flask

Định nghĩa bài toán

- **Đầu vào:** Video giao thông từ camera cố định và vạch đếm L được cấu hình trên khung hình.

$$[V = I_1, I_2, \dots, I_T]$$

- **Đầu ra:** Số lượt phương tiện đi qua vạch theo loại phương tiện, chiều di chuyển và khoảng thời gian:

$$\left[\widehat{N}_{c,d,\Delta} = \sum_{e_k \in \mathcal{E}} F\left[\hat{c}_k = c, d_k = d, \tau_k \in \Delta\right]\right]$$

Trong đó: c là loại phương tiện; d là chiều di chuyển; Δ là khoảng thời gian; $\mathcal{E}$ là tập các lượt qua vạch được hệ thống ghi nhận.

- **Mục tiêu:** Xây dựng hệ thống phát hiện, phân loại và theo dõi phương tiện để ước lượng $\widehat{N}_{c,d,\Delta}$ gần với số lượt thực tế, đồng thời đáp ứng yêu cầu về tốc độ xử lý.
- **Các loại phương tiện:** Xe máy · Ô tô · Xe buýt · Xe tải.

Kiến trúc tổng thể hệ thống

KHỐI XỬ LÝ AI

Video đầu vào	Trích khung hình	Phát hiện (YOLOv8n)	Theo dõi (ByteTrack)	Đếm (Line-crossing)	Tổng hợp	Lưu trữ (SQLite)	Dashboard (Flask)
Camera cố định / file .mp4	OpenCV đọc từng frame	bbox + nhãn lớp	gán ID duy nhất	xác định hướng	theo loại & thời gian	ghi log theo thời gian	biểu đồ realtime

CÔNG CỤ / THƯ VIỆN CHÍNH

OpenCV

Đọc video, trích frame, hình học vạch cắt

Ultralytics YOLOv8 + ByteTrack

Phát hiện & theo dõi đối tượng

SQLite Engine

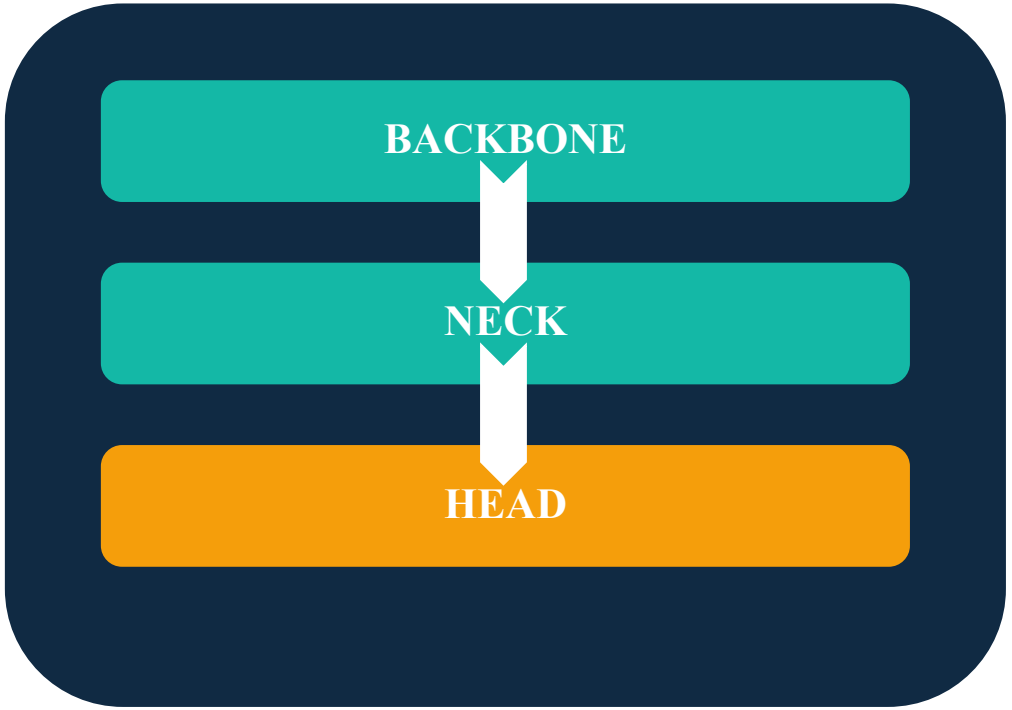
Truy vấn SQL, lưu trữ nhẹ, 1 file .db

Flask + Docker

Giao diện web, đóng gói triển khai

Mô hình phát hiện YOLOv8n

- Kiến trúc one-stage: định vị và phân loại trong một lần suy luận.
- Ba thành phần: backbone, neck và head; dự đoán anchor-free.
- Phiên bản nano có quy mô nhỏ, phù hợp xử lý video liên tục.
- Hiệu năng được nâng bằng học chuyển giao và tinh chỉnh theo miền dữ liệu.



3.0 M
tham số

Học chuyển giao và hai lộ trình tinh chỉnh



Theo dõi và đếm phương tiện



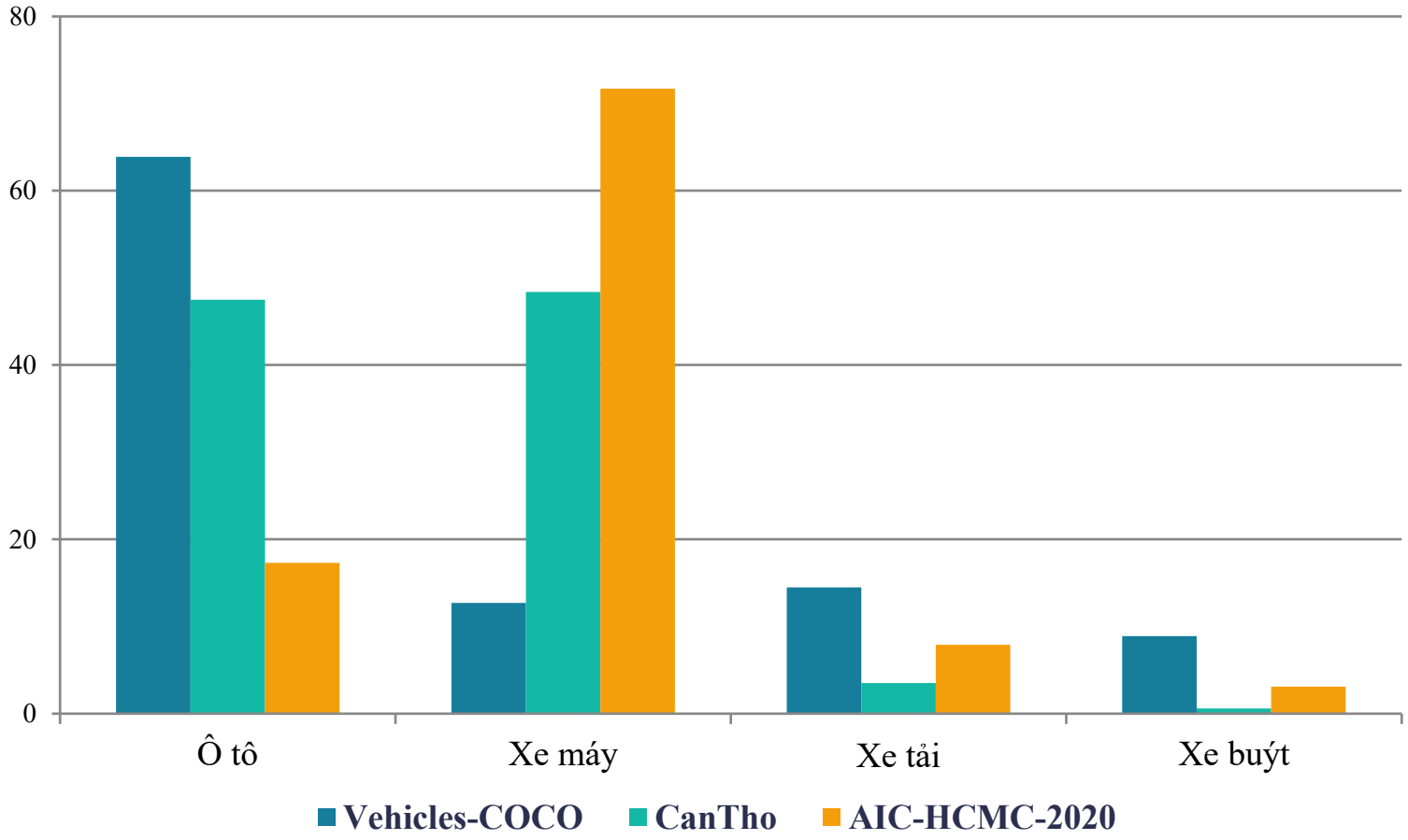
- **ByteTrack liên kết kết quả phát hiện và duy trì mã định danh qua các khung hình.**
- **Majority voting chuẩn hóa nhãn khi xảy ra class switching.**
- **Line-crossing dùng đổi dấu tích có hướng để xác định lượt cắt và chiều di chuyển.**
- **Mỗi ID chỉ được ghi nhận một lần nhằm tránh đếm trùng.**

Ba bộ dữ liệu và vai trò

Vehicles-COCO	CanTho	AIC-HCMC-2020
18.998 ảnh	1.110 ảnh	15.104 ảnh
Nền tảng tổng quát	Miền trung gian	Miền đích TP.HCM

Cùng 4 lớp nhãn • Chia dữ liệu 70 : 15 : 15

Mất cân bằng lớp giữa các miền



Điểm nổi bật

- COCO thiên về ô tô
- HCMC có 71,7% xe máy
- Xe buýt là lớp thiểu số
- Phân bố miền khác biệt rõ

Thiết kế thực nghiệm và tiêu chí đánh giá

Phát hiện

Precision • Recall • mAP@0.5 • mAP@0.5:0.95

Chéo miền

Ma trận 3×3 • mức suy giảm hiệu năng

Ablation

Lộ trình A/B • 3 seed: 42, 1407, 1712

Tracking

Class switching • đồng thuận nhãn • phân mảnh ID

Đếm

Đối chiếu thủ công • sai số tuyệt đối

Hệ thống

FPS trung bình • độ ổn định vận hành

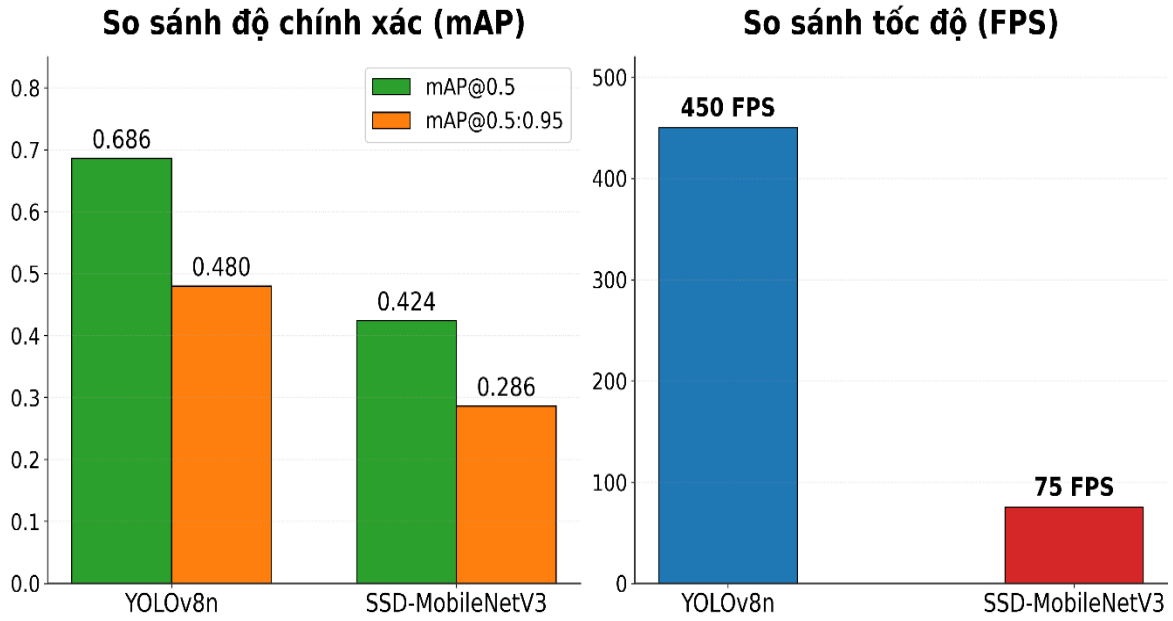
Kết quả Baseline và đối chứng kiến trúc

0,686
YOLOv8n mAP@0.5

0,424
SSD-MobileNetV3 mAP@0.5

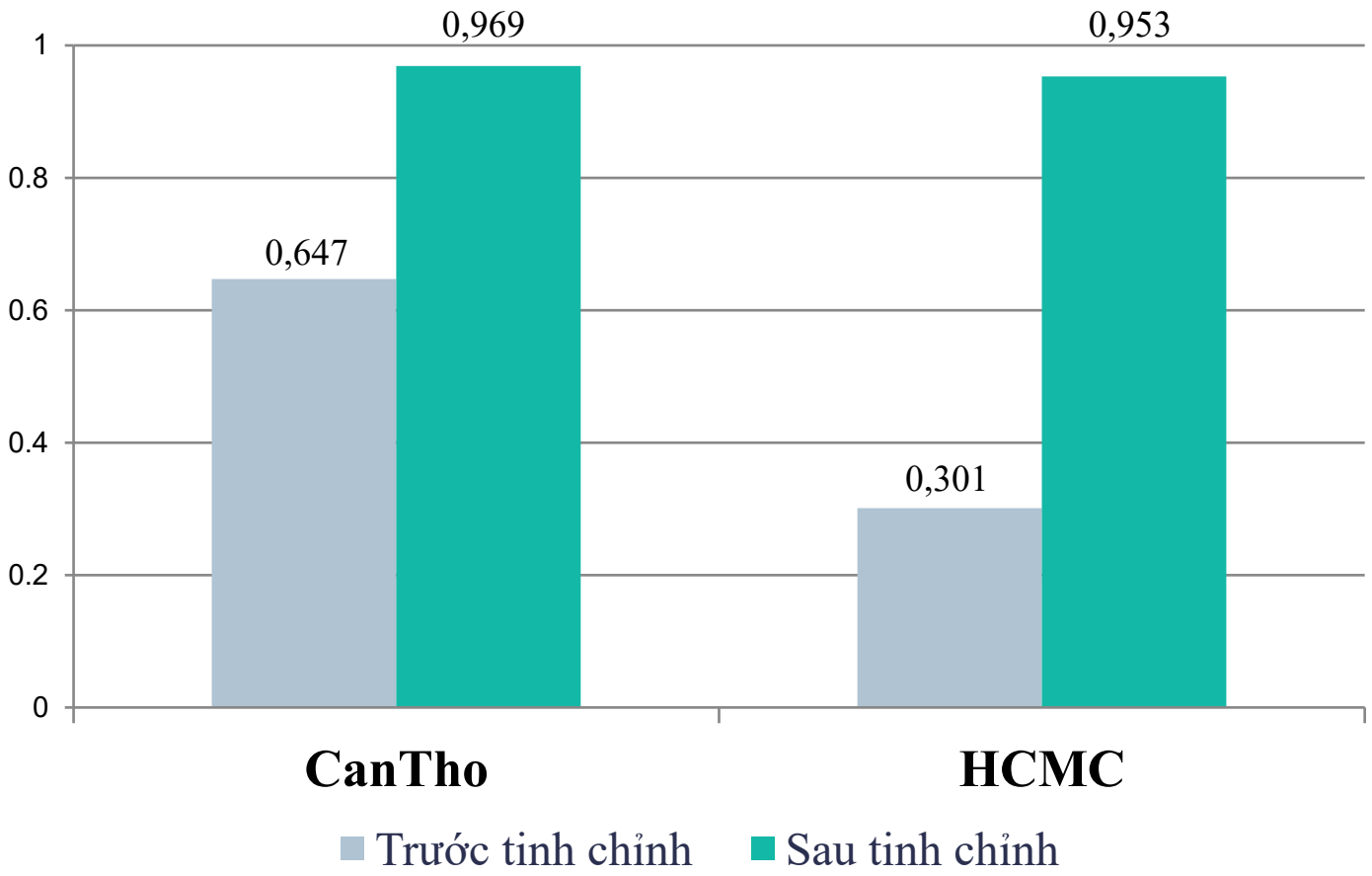
~450
FPS YOLOv8n

~75
FPS SSD-MobileNetV3



Kết luận: YOLOv8n được chọn làm kiến trúc phát hiện chính.

Hiệu quả tinh chỉnh theo miền dữ liệu



mAP@0.5

+49,8%
trên CanTho

> 3 lần
trên HCMC

Đánh giá chéo miền: thích nghi và quên kiến thức

Mô hình / Tập kiểm thử	COCO	CanTho	HCMC
Baseline COCO	0,686	0,647	0,373
Fine-tune CanTho	0,176	0,969	0,362
Fine-tune HCMC	0,084	0,750	0,953

SUY GIẢM

74,3%

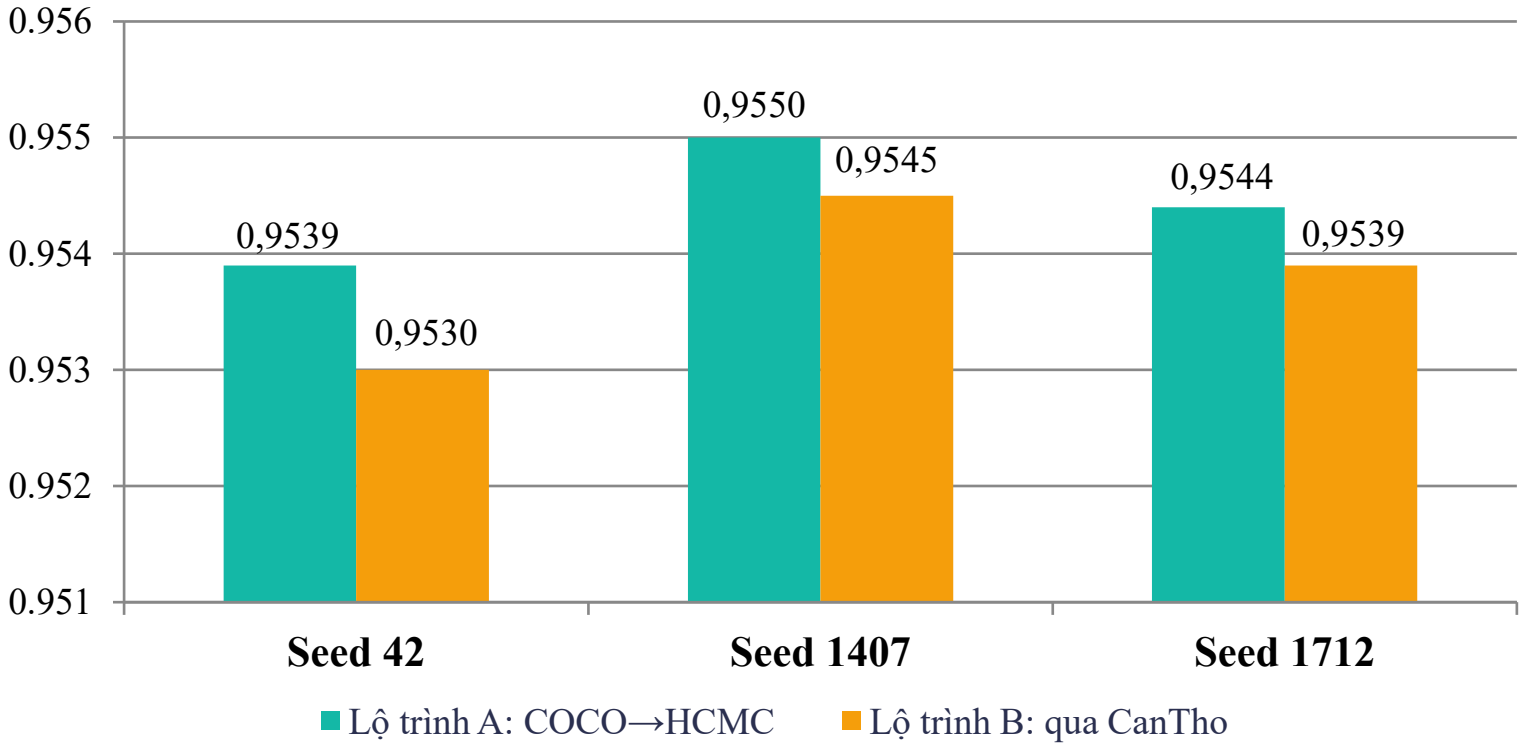
COCO sau bước CanTho

22,6%

CanTho sau bước HCMC

Tinh chỉnh cải thiện mạnh trên miền đích nhưng làm suy giảm tri thức ở miền trước.

Ablation study: CanTho có cần thiết không?



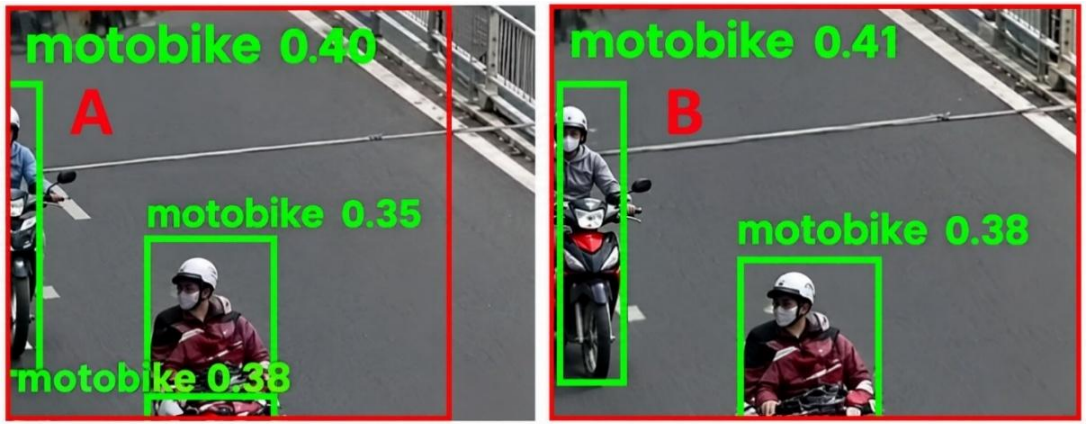
CHÊNH LỆCH

0,0005 đến 0,0009

CanTho không cải thiện đáng kể hiệu năng trên HCMC.

Chọn lộ trình A làm mô hình chính thức theo nguyên tắc tối giản.

Kết quả theo dõi đối tượng



97,70%
đồng thuận nhãn sau majority voting

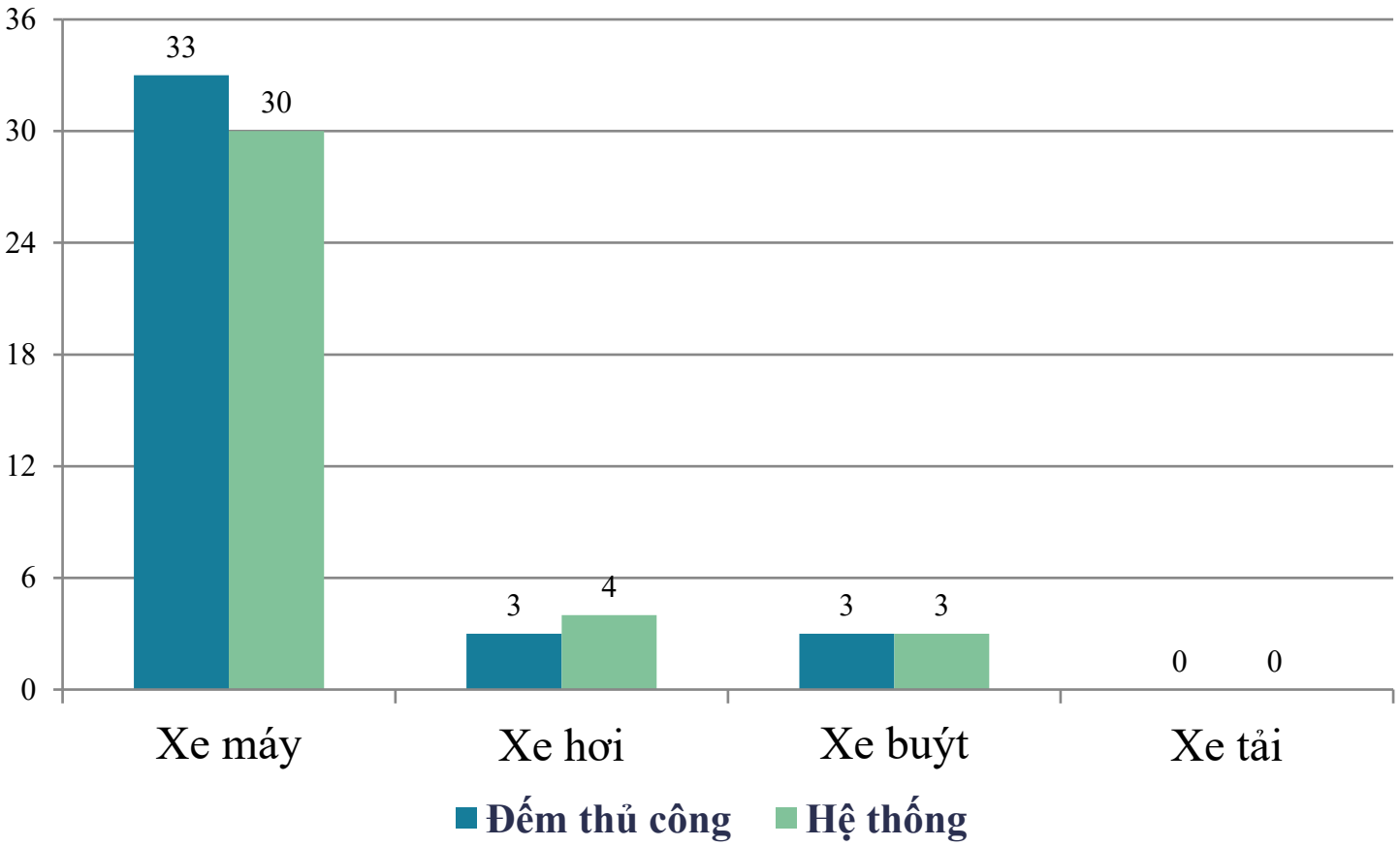
13,3%
ID có thay đổi nhãn

15
ID hợp lệ

2 cụm
phân mảnh sau hợp nhất bắc cầu

Nút thắt chính: mất dấu tạm thời có thể tạo ID mới và ảnh hưởng kết quả đếm.

Độ chính xác đếm phương tiện



89,7%

độ chính xác tổng thể

Sai lệch: thiếu 3 xe máy, thừa 1 xe hơi.

Dashboard và kiến trúc triển khai

Trình duyệt người dùng

HTTP / JSON

CONTAINER DOCKER

Ứng dụng Flask

- Xử lý yêu cầu HTTP / giao diện & quản lý phiên làm việc
- Điều phối xử lý AI thời gian thực (thành phần VideoWorker)

Volume mount *Volume mount* *Volume mount*

/app/data
Cơ sở dữ liệu

/app/videos
Video nguồn

/app/model
Mô hình .pt

Hiệu năng hệ thống và Dashboard



26,39
FPS trung bình

26,55
FPS trung vị

20,10
FPS thấp nhất

1,24
độ lệch chuẩn FPS

Hệ thống vận hành liên tục, tốc độ dao động hẹp và không phát sinh gián đoạn trong phiên đo.

Tổng hợp kết quả theo câu hỏi nghiên cứu

Câu hỏi	Trọng tâm	Bằng chứng chính	Kết luận
Q1	Tinh chỉnh HCMC	mAP@0.5 đạt khoảng 0,954	Được xác nhận
Q2	Vai trò CanTho	Chênh dưới 0,001 qua 3 seed	Không cần cho mô hình chính thức
Q3	Tổng quát chéo miền	Suy giảm 74,3% và 22,6%	Có hiện tượng quên kiến thức
Q4	Hệ thống hoàn chỉnh	89,7% • 26,39 FPS	Khả thi trong điều kiện khảo sát

Kết luận

- Xây dựng thành công chuỗi phát hiện, theo dõi, đếm, lưu trữ và trực quan hóa phương tiện.
- YOLOv8n phù hợp hơn SSD-MobileNetV3 về độ chính xác và tốc độ.
- Tinh chỉnh trực tiếp COCO→HCMC được chọn làm lộ trình chính thức, CanTho giữ vai trò đối chứng.
- Đánh giá chéo miền cho thấy thích nghi miền đích đi kèm quên kiến thức ở miền trước.
- Hệ thống đạt độ chính xác đếm 89,7% và tốc độ trung bình 26,39 FPS trong điều kiện khảo sát.

Hạn chế và hướng phát triển

Phân mảnh ID	Hợp nhất track bằng thời gian, vị trí và đặc trưng ngoại hình
Quy mô kiểm chứng nhỏ	Tăng số seed, áp dụng khoảng tin cậy và kiểm định giả thuyết
Quên kiến thức	Học liên tục, replay dữ liệu miền gốc, chưng cất tri thức
Đánh giá đếm còn ngắn	Nhiều video, thời lượng dài, hai hướng và nhiều khung giờ
Một camera, chưa edge	Đa camera, lượng tử hóa, cắt tỉa và kiểm thử ban đêm/mưa

Tài liệu tham khảo chính

- [1] Ribeiro & Hirata (2023). Combining YOLO and visual rhythm for vehicle counting.
- [2] Khan & Rishe (2026). Unified vehicle detection, tracking and counting using knowledge distillation.
- [3] Nguyễn Mạnh Cường và cộng sự (2021). Phân loại phương tiện giao thông dựa trên thị giác máy tính.
- [4] Nguyễn Văn Viết (2023). Đếm phương tiện giao thông sử dụng dữ liệu lớn thời gian thực và học sâu.
- [5] Redmon et al. (2016). You Only Look Once. CVPR.
- [7] Zhang et al. (2022). ByteTrack. ECCV.
- [8] Wojke et al. (2017). DeepSORT. ICIP.
- [9] Liu et al. (2016). SSD. ECCV.
- [10] Howard et al. (2019). MobileNetV3. ICCV.
- [11–14] Tài liệu chính thức: SQLite, Flask, OpenCV, Docker.

XIN CẢM ƠN • Q&A